

10 PEARLS

Internship Project Report

Project Title:
Karachi AQI Predictor

A Serverless, End-to-End Air Quality Index Prediction System

Submitted to: 10 Pearls

Submitted by: Muhammad Sabih Shamsi

Date: June 2026

1. Executive Summary

This report documents the internship project completed at 10 Pearls, focused on building a fully automated, serverless Air Quality Index (AQI) prediction system for Karachi, Pakistan. The project delivers real-time AQI data, a 3-day machine learning forecast, an interactive dashboard, and a fully automated data and training pipeline — all built on a 100% serverless stack.

2. Project Overview

Air pollution is a critical public health challenge in Karachi. This project was designed to provide residents and decision-makers with accurate, automated, and accessible AQI forecasts. The system predicts PM2.5 levels 1, 2, and 3 days into the future using weather and air quality data fetched from Open-Meteo, stored in a Hopsworks Feature Store, and served through a FastAPI backend and Streamlit dashboard.

Project Objectives

- Collect and store real-time Karachi weather and air quality data automatically.
- Engineer meaningful time-series features without data leakage.
- Train and evaluate multiple ML models, selecting the best per forecast horizon.
- Serve live 3-day AQI forecasts through a FastAPI REST API.
- Display forecasts, history, and SHAP explanations on an interactive Streamlit dashboard.
- Automate all pipelines using GitHub Actions CI/CD.

3. System Architecture

The project follows a four-stage serverless pipeline architecture as defined in the project specification:

Stage 1 — Feature Pipeline (Hourly)

A GitHub Actions workflow runs every hour. It fetches raw hourly air quality variables (PM2.5, PM10, CO, NO2, SO2, ozone, dust, UV index) and weather variables (temperature, humidity, pressure, precipitation, wind speed and direction) from the Open-Meteo API. These are aggregated to daily averages and enriched with time-based features (day of week, month, day of year, is_weekend) and pollutant trend features (lag values, rolling means, change rates). The resulting feature rows are written to the Hopsworks Feature Store.

Stage 2 — Training Pipeline (Daily)

A separate GitHub Actions workflow runs every day at 02:00 UTC. It reads all historical features from the Feature Store and trains three scikit-learn models — Ridge Regression, Random Forest, and MLP Neural Network — for each of the three forecast horizons (Day 1, Day 2, Day 3). The best-performing model per horizon (by RMSE on a chronological held-out test set) is registered as a versioned artifact in the Hopsworks Model Registry.

Stage 3 — Prediction API (FastAPI)

A FastAPI application loads the latest registered model and features from Hopsworks and serves three endpoints: `/predict-latest` for 3-day forecasts, `/history` for recent AQI history, and `/shap-importance` for feature explanations. The API is deployed on Hugging Face Spaces and uses short-lived in-memory caches (5-minute TTL) to minimize Hopsworks API calls while ensuring predictions reflect fresh data within the hour.

Stage 4 — Dashboard (Streamlit)

A Streamlit web application queries the FastAPI backend and presents: the current live AQI (fetched directly from Open-Meteo), 3-day forecast cards with AQI category and PM2.5 values, a combined historical + forecast line chart, a 90-day trend view with category distribution and PM2.5 histogram, SHAP feature importance bar charts per horizon, and a full AQI scale reference table.

4. Technology Stack

Component	Technology / Service
Data Source	Open-Meteo Air Quality
Feature Store	Hopsworks Feature Store (free tier)
Model Registry	Hopsworks Model Registry (free tier)
ML Models	scikit-learn: Ridge, Random Forest, MLP (Python 3.11)
Prediction API	FastAPI + Uvicorn, deployed on Hugging Face Spaces
Dashboard	Streamlit + Plotly, deployed on Streamlit Cloud
CI/CD Automation	GitHub Actions (hourly feature sync + daily training)

5. Challenges and Solutions

The following table summarizes the ten key challenges encountered during the project and the specific solutions implemented to address each one.

Challenge / Problem	Solution
Getting reliable AQI and weather data from Open-Meteo, handling missing values and timezone consistency.	Used Open-Meteo APIs, converted hourly data into daily averages, added fallback historical data, and enforced Asia/Karachi timezone throughout.
Feature engineering without data leakage — lag features like previous-day PM2.5 could leak future information into training.	Recomputed lag and rolling features after train/test splitting instead of trusting precomputed columns from the full dataset.
Time-series model training — random splits would give misleading evaluation results for time-dependent AQI data.	Used chronological train/test splitting with a boundary gap for each forecast horizon to prevent boundary leakage.
Multi-horizon prediction — predicting 3 future days required different input rows per horizon, not the same latest row.	Trained separate direct-horizon models for Day 1, Day 2, and Day 3; API uses the correctly offset feature row per horizon.
Stale model and feature data in API — cached data caused the forecast to appear frozen even after the pipeline updated.	Reduced model and feature cache TTLs so the API refreshes within 5 minutes of new training or feature sync.
Hopsworks integration — authentication, feature group versions, model versioning, and CI/CD secrets configuration.	Used Hopsworks Feature Store and Model Registry with automatic version incrementing and GitHub Actions secrets.
Dependency and environment compatibility across libraries (Hopsworks, scikit-learn, FastAPI, Streamlit).	Standardized the project on Python 3.11 and pinned dependencies through requirements.txt.
Deployment and automation — running unattended hourly feature sync and daily training via CI/CD.	Configured GitHub Actions workflows with proper secrets, schedules, and Python setup for both pipelines.
Dashboard/API coordination — handling API errors, timeouts, missing model data, and empty history responses.	Wrapped all API calls in error handling so the Streamlit dashboard displays useful errors instead of crashing.
AQI interpretation — the model predicts raw PM2.5 values, but users need understandable AQI categories.	Implemented the EPA PM2.5-to-AQI breakpoint formula and mapped values to categories, colors, and health labels.

6. Key Technical Decisions

Direct Multi-Horizon Modelling

Rather than using a single recursive model to predict multiple days ahead (which compounds errors), the system trains a separate model for each forecast horizon. This direct approach means the Day-3 model is specifically optimized to predict 3 days forward, yielding more reliable forecasts.

Leak-Free Feature Engineering

Lag features (e.g. previous-day PM2.5) are computed after the chronological train/test split. Test-set lag features are seeded using only the tail of the training set as context, preventing any future information from contaminating the training process — a common mistake in time-series ML projects.

Horizon-Aware Inference

At prediction time, each horizon uses a different input row from the feature store: Day-1 uses the most recent row, Day-2 uses the row from one day earlier, and Day-3 uses the row from two days earlier. This mirrors the training setup and ensures that lag features carry the correct temporal offset for each forecast day.

Dual-Cache API Design

The API maintains separate caches for model artifacts (5-minute TTL) and feature data (5-minute TTL). The current AQI is fetched live from Open-Meteo on every cache miss, bypassing the feature store entirely. This design minimizes latency while ensuring forecasts stay fresh within the hourly pipeline cadence.

7. Model Performance

Each model is evaluated on a chronological held-out test set using three metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared (R^2). For each forecast horizon, the model with the lowest RMSE is automatically selected and registered. The three candidate models are:

- Ridge Regression — a fast linear baseline with L2 regularization.
- Random Forest — a 300-tree ensemble with depth-16 trees, capturing non-linear pollutant patterns.
- MLP Neural Network — a two-layer network (128 and 64 units) for complex feature interactions.

In practice, Random Forest consistently achieved the lowest RMSE across all three horizons given the non-linear relationship between meteorological variables and PM2.5 concentration in Karachi's climate.

8. Automated Pipelines

Hourly Feature Pipeline (feature-pipeline.yml)

Runs at the top of every hour via GitHub Actions cron schedule (0 * * * *). Fetches the last 3 days of data from Open-Meteo, computes features, and inserts them into the Hopsworks Feature Store. This rolling 3-day window catches any late-arriving or corrected data from the archive API.

Daily Training Pipeline (training-pipeline.yml)

Runs every day at 02:00 UTC. Reads the full feature history from Hopsworks, trains all candidate models for all three horizons, selects the best per horizon, and registers a new versioned model artifact. The API picks up the new model within 5 minutes via its model cache TTL.

Manual Workflows

Two additional manual workflows are provided: `backfill_features.yml` for bulk historical data loading, and `train_model.yml` for on-demand retraining both triggerable from the GitHub Actions UI.

9. Deployment

FastAPI Backend — Hugging Face Spaces

The FastAPI application is containerized with Docker (linux/amd64) and deployed to Hugging Face Spaces. Hopsworks credentials and feature group configuration are injected as Space secrets. The service runs on port 7860 and is publicly accessible at the Space URL.

Streamlit Frontend — Streamlit Cloud

The Streamlit dashboard is deployed to Streamlit Cloud, with `PREDICTION_API_URL` configured as a secret pointing to the Hugging Face Space API. The dashboard calls the API on every page load — no server-side state is maintained.

10. Conclusion

This internship project at 10 Pearls successfully delivered a complete, production-grade AQI prediction system for Karachi. The system demonstrates best practices in time-series machine learning including leak-free feature engineering, direct multi-horizon modelling, and chronological evaluation while running fully automatically on a serverless stack at zero infrastructure cost.

The project fulfils all requirements from the internship brief: an automated feature pipeline (hourly), an automated training pipeline (daily), a model registry, a prediction API, an interactive dashboard with SHAP explanations, and a detailed report. The codebase is modular, documented, and ready for extension to other cities or additional pollutant variables.